

Intelligent Robotics Summary

[Notes by NotebookLM] [Chinello Alessandro and Piai Luca]

Academic Year 2025-2026

Contents

1	Introduction to robotics	1
1.1	The Evolution of Robotics and ROS 2 Framework	1
1.1.1	The ROS 2 Framework	1
1.2	The Four Industrial Revolutions	2
1.3	The Failure of “Dark Factories” and the Need for Humans	2
1.3.1	The Concept	2
1.3.2	The Limitation	3
1.3.3	The Solution	3
1.4	Industry 5.0 and Collaborative Robotics	3
1.4.1	Industry 5.0	3
1.4.2	Collaborative Robots (Cobots)	3
1.5	Historical Lineage: Tools vs. Agents	3
1.6	History and Market Realities	4
1.7	Autonomy, Intelligence, and Locomotion	4
1.7.1	Autonomy vs. Automation	4
1.8	Cognitive Architectures	5
2	Locomotion	7
2.1	Fundamentals of Locomotion	7
2.1.1	Nature vs. Technology	7
2.1.2	Efficiency of Motion	7
2.2	Legged Locomotion and Stability	8
2.2.1	Why Legs?	8
2.2.2	Stability Types	8
2.2.3	Passive Dynamic Walkers	8
2.2.4	Humanoid Challenges	8
2.3	Wheeled Locomotion and Kinematics	8
2.3.1	Fundamentals of Wheeled Locomotion	8

2.4	Robot Configurations and Maneuverability	9
2.5	Stability and Environment Interaction	10
3	Kinematics of mobile robots	11
3.1	Kinematics and Coordinate Frames	11
3.1.1	Reference Frames	11
3.1.2	Motion Constraints (Holonomic vs. Non-Holonomic)	11
3.1.3	Dead Reckoning	11
3.2	Differential Drive Math	12
4	Robot Perception	13
4.1	Robot Perception and Sensors	13
4.1.1	The Challenge of Perception	13
4.2	The Perception Pipeline: From Data to Semantics	13
4.3	Sensor Taxonomy	14
4.3.1	Proprioceptive vs. Exteroceptive	14
4.3.2	Active vs. Passive	14
4.4	Characterizing Sensor Performance	15
4.4.1	Key Metrics	15
4.4.2	Accuracy vs. Repeatability	15
4.4.3	Industrial Application	15
4.5	Motion and Heading Sensors (Dead Reckoning)	16
4.6	Tactile and Contact Sensors	16
4.7	Technology Drivers	17
4.8	Advanced Proprioception and Exteroceptive Sensing	17
4.8.1	Inertial Measurement and Dead Reckoning	17
4.8.2	Contact and Tactile Sensors	17
4.8.3	Proximity Sensors (Infrared)	18
4.8.4	Triangulation and Structured Light	19
4.8.5	Time of Flight (ToF) Sensors	19
4.9	Sensor Technologies in Mobile Robotics (Paper 1)	20
4.9.1	Definition and Classification	20
4.9.2	Basic Components (Transducers)	20
4.9.3	Proprioceptive and Contact Sensors	21
4.9.4	Global Localization Systems	21
4.9.5	Inertial and Heading Sensors (IMU)	22
4.9.6	Vision Systems (Cameras)	22

4.9.7	Ranging and Depth Sensors	22
5	Localization and mapping	25
5.1	Mapping, Navigation, and Advanced Sensing	25
5.1.1	Advanced Range Sensing (Lidar, Radar, GPS)	25
5.2	The Mapping Problem: Metric vs. Topological	26
5.2.1	Metric Maps (Quantitative)	26
5.2.2	Topological Maps (Qualitative)	26
5.2.3	Hybrid Maps	26
5.3	Topological Navigation and Landmarks	26
5.3.1	Landmarks	26
5.3.2	Distinctive Places & Hill Climbing	27
5.3.3	Symbol Grounding	27
5.4	Configuration Space (C-Space)	27
5.5	3D Mapping Representations	28
5.6	Simultaneous Localization and Mapping (SLAM)	28
5.6.1	The Hierarchy of Localization Problems	28
5.6.2	The SLAM Problem Definition	29
5.7	The Probabilistic Approach	29
5.7.1	Bayes Filter	29
5.8	The Kalman Filter (KF) and Extended Kalman Filter (EKF)	29
5.8.1	Kalman Filter	29
5.8.2	Extended Kalman Filter (EKF)	30
5.9	Particle Filters	30
5.10	Solving the SLAM Problem	30
5.10.1	EKF SLAM	31
5.10.2	Particle Filter SLAM (FastSLAM)	31
5.10.3	Graph-based SLAM	31
6	Motion Planning	33
6.1	Motion Planning Fundamentals	33
6.2	Human-Robot Collaboration (HRC) Planning	34
6.3	Geometric Motion Planning and Algorithms (Paper 2)	35
6.3.1	The Geometric Path Planning Problem	35
6.3.2	Complexity and Completeness	35
6.3.3	Sampling-Based Planning	36
6.3.4	Combinatorial and Potential Field Approaches	37

6.3.5	Planning with Differential Constraints	37
6.3.6	Extensions and Variations	38
7	Software Architectures for robotics and robotics paradigms	39
7.1	Robot Architectures	39
7.1.1	The Hierarchical Paradigm (Classic AI)	39
7.1.2	The Reactive Paradigm (Behavior-Based Robotics)	40
7.1.3	The Hybrid Paradigm	41
7.1.4	The Hybrid Paradigm	41
8	Human Robot-interaction	43
8.1	Teleoperation, and Shared Intelligence	43
8.1.1	Teleoperation and Telesystems	43
8.1.2	Taxonomy of Control and Interaction	43
8.1.3	The “Horse Metaphor” (H-Metaphor)	44
8.1.4	Brain-Machine Interfaces (BMI) and Neuro-Robotics	44

Chapter 1

Introduction to robotics

1.1 The Evolution of Robotics and ROS 2 Framework

1.1.1 The ROS 2 Framework

Purpose and Definition

ROS (Robot Operating System) is not an actual operating system but a framework or middleware that sits on top of an OS (usually Linux). Its primary goal is to facilitate the building of robot software using reusable modules, preventing developers from having to “reinvent the wheel” for every new robot.

Modularity

Success in robotics software relies on modularity. This allows developers to use code written by others (e.g., using a localization module from another lab) while focusing on their specific application.

Key Components

- **Nodes:** These are the atomic units of processing. A node is a process that performs computation, such as image processing or wheel control. Ideally, each node should handle a single specific functionality.
- **Topics:** These act as communication channels or a “whiteboard” where nodes exchange information. A node can publish messages to a topic or subscribe to read them. Topics are strictly for communication, not for configuration.

Tools and Structure

- **Workspaces:** Directories used to organize and build ROS packages.
- **ROS 2 Doctor:** A command-line tool (`ros2 doctor`) used to check if the environment is correctly set up and to diagnose issues.

1.2 The Four Industrial Revolutions

The history of robotics is tied to the evolution of industrial production:

- **Industry 1.0 (Mechanization):** Began in the late 1700s, characterized by the use of steam and water power to activate mechanical machines.
- **Industry 2.0 (Mass Production):** Focused on work organization and the introduction of electricity. A key innovation was the assembly line (inspired by slaughterhouses), which increased standardization and productivity by having workers perform repeated specific tasks.
- **Industry 3.0 (Automation):** Introduced electronics, PLCs (Programmable Logic Controllers), and software. This era saw the rise of traditional industrial robots used for repetitive, dirty, or dangerous jobs (like painting cars to avoid toxic fumes). These robots follow fixed programs and require fences for safety.
- **Industry 4.0 (The Smart Factory):** Represents the shift from mass production to custom-based production. It relies on cyber-physical systems, machine learning, big data, and cloud technology to create interoperable, reprogrammable systems that can adapt to different product needs (e.g., different car configurations) without halting production.

1.3 The Failure of “Dark Factories” and the Need for Humans

1.3.1 The Concept

With Industry 4.0, some companies (like Tesla and Amazon) attempted to create “dark factories”—fully automated facilities that require no humans, and therefore no lighting or heating.

1.3.2 The Limitation

These attempts often failed because the real world is too complex and dynamic for current robotics technology. Robots struggle with unmodeled events (e.g., a foreign object on the floor) or mechanical failures that require human dexterity and reasoning to fix.

1.3.3 The Solution

The industry realized that collaborative robotics is more effective. Humans are needed for supervision, high-level reasoning, and dexterity, while robots handle heavy lifting and repetitive tasks.

1.4 Industry 5.0 and Collaborative Robotics

1.4.1 Industry 5.0

A new paradigm proposed (especially by the EU) to complement Industry 4.0. It moves beyond efficiency to focus on sustainability, resilience, and a human-centric approach. It emphasizes the well-being of the worker, partly to attract employees in Western countries where the workforce is aging and shrinking.

1.4.2 Collaborative Robots (Cobots)

Unlike Industry 3.0 robots, cobots are designed to work alongside humans without protective fences.

- **Safety:** They are safe by design, equipped with sensors to stop upon contact or detection of humans, unlike older robots that would continue moving regardless of obstacles.
- **Benefits:** They reduce the factory footprint (no cages needed) and are cost-effective, making them ideal for Small and Medium Enterprises (SMEs).
- **Interaction:** Advanced collaborative robots may use displays to indicate their future actions (e.g., gaze direction) to help humans predict their movements.

1.5 Historical Lineage: Tools vs. Agents

Robotics evolved along two distinct parallel tracks that are now converging:

- **The “Tool” Track (Industrial):** Driven by the nuclear industry’s need to handle radioactive materials remotely. This focused on precision and teleoperation, leading to the Unimate (1959), the first industrial robot, which was used in foundries.
- **The “Agent” Track (Intelligent):** Driven by the space race and AI research (e.g., DARPA, NASA). This focused on autonomy and reasoning. A key milestone was Shakey (late 1960s), the first mobile robot able to perceive its environment, reason, and plan actions to achieve a goal.

Convergence: Modern robotics combines these tracks. Today’s collaborative robots merge the reliability and strength of industrial “tools” with the sensing and adaptability of intelligent “agents”.

1.6 History and Market Realities

- **First Commercial Robot:** The Unimate (1959, founded 1956) was the first industrial robot, not the Roomba or machines by Turing.
- **The Roomba Success Story:** Early vacuum robots failed because engineers tried to make them perfect (expensive sensors, mapping) costing $\approx$ \$2,000. iRobot designed the Roomba based on what people would spend on an impulse (\$250). It used random motion and simple behaviors rather than complex planning, proving that “good enough” autonomy can be commercially superior.
- **Artificial Intelligence Term:** The term was coined by John McCarthy in 1956, not Turing.

1.7 Autonomy, Intelligence, and Locomotion

1.7.1 Autonomy vs. Automation

There is a distinct difference between automation and autonomy in robotics, defined by how the system interacts with its environment and data.

Automation (The Tool)

- **Definition:** Automation refers to physically situated tools designed to perform highly repetitive, pre-planned actions.

- **Characteristics:** It relies on signals (e.g., a sensor reading a voltage) rather than symbols. It operates under the **Closed World Assumption**, meaning the environment is perfectly modeled, nothing relevant changes, and there are no surprises.
- **Decision Making:** Decisions are deterministic and handled at the signal level (e.g., if voltage is high, stop).
- **Goal:** To create stable control loops for well-defined tasks.

Autonomy (The Agent)

- **Definition:** Autonomy is the state of being “self-governing”. In engineering, this does not mean “free will” or moral independence, but the mechanical ability to regulate oneself using feedback loops.
- **Historical Example:** James Watt’s Flyball Governor is an early example of autonomy. It mechanically regulated steam flow to keep engine speed constant without human intervention, effectively adapting to changes in load.
- **Characteristics:** It operates under the **Open World Assumption**, where the model of the world is partial, and the robot must adapt to dynamic, unmodeled events (e.g., a rock on the path).
- **Decision Making:** It uses symbols (abstract representations like “cat” or “door”) and reasoning to generate new plans rather than just executing pre-written ones.

The “Monkey and Banana” Problem

This classic AI problem illustrates the failure of closed-world logic in the real world. A planner might deduce how to reach bananas using a cart, but in the real world (Open World), unmodeled constraints (like the bananas being tied to the cart) can cause the plan to fail. This necessitates perception and adaptation.

1.8 Cognitive Architectures

How a robot organizes its “brain” to connect sensing to acting:

- **Deliberative Architecture (Sense-Plan-Act):** The robot perceives the world, builds a model, plans an action, and then acts. This is slow and thoughtful (like a chess player).

- **Reactive Architecture (Sense-Act):** A direct link between stimulus and response, similar to reflexes in animals (e.g., a cockroach running from light). It is fast but has no memory or long-term planning.
- **Hybrid Architecture:** The standard for modern robotics. It combines a Deliberative layer (upper brain) for long-term planning with a Reactive layer (spinal cord/lower brain) for immediate safety and reflexes.

Chapter 2

Locomotion

2.1 Fundamentals of Locomotion

Locomotion is the physical interaction between a vehicle and its environment to move from place to place.

2.1.1 Nature vs. Technology

Nature never developed the wheel or the fully rotating joint. Roboticists often use bio-inspiration (mimicking geckos, fish, birds), but direct imitation is difficult because robots use gears and motors, not muscles.

2.1.2 Efficiency of Motion

- **Rolling:** (hard wheels on hard surfaces, like trains) is the most energy-efficient form of motion.
- **Walking:** Surprisingly efficient because it is mechanically similar to rolling (the center of mass moves in a way that mimics a rimless wheel).
- **Inefficient Modes:** Sliding, crawling, and running require significantly more power.

2.2 Legged Locomotion and Stability

2.2.1 Why Legs?

Unlike wheels, legs can overcome obstacles higher than their radius, step over gaps, and keep the payload (body) level while climbing slopes.

2.2.2 Stability Types

- **Static Stability:** The center of gravity stays within the polygon of support (requires at least 3 legs, or large feet).
- **Dynamic Stability:** The robot is constantly “falling” and catching itself (like a human walking). This allows for 2-legged walking but requires active control.

2.2.3 Passive Dynamic Walkers

A class of robots that can walk down a slope without any motors or control, solely powered by gravity and inertia (converting potential to kinetic energy). This proves that walking can be naturally efficient.

2.2.4 Humanoid Challenges

- **Energy Cost:** Robots like Honda’s ASIMO are inefficient (high Cost of Transport) because they walk with bent knees to maintain stability, requiring constant motor torque.
- **ZMP (Zero Moment Point):** A control algorithm used to ensure stability by keeping the moment forces at the foot contact point at zero.
- **State of the Art:** While Boston Dynamics shows impressive agility, many humanoid demonstrations are still tele-operated or highly rehearsed (open loop), lacking true reliable autonomy for real-world deployment.

2.3 Wheeled Locomotion and Kinematics

2.3.1 Fundamentals of Wheeled Locomotion

While legged locomotion is bio-inspired, wheeled locomotion is the dominant form for mobile robots due to energy efficiency on hard surfaces. However, every wheel added to

a robot imposes constraints on its motion.

Degrees of Freedom (DOF)

- **Standard Wheel:** Has two degrees of freedom. It can rotate (spin) around its horizontal axis and pivot around the point of contact with the ground. However, it generally does not allow sideways sliding (lateral motion) due to friction.
- **Castor Wheel:** Consists of a wheel with a vertical pivoting axis offset from the wheel's center. It effectively has three degrees of freedom (spinning, pivoting offset, and rotation around the contact point), allowing it to align with the direction of motion.
- **Swedish (Mecanum) Wheel:** Features rollers set at an angle (usually 45°) around the main wheel perimeter. It has three degrees of freedom because the rollers allow the wheel to move sideways (laterally) without sliding friction, enabling omnidirectional movement.
- **Spherical Wheel:** A ball held in a socket. It has three degrees of freedom and poses no constraints on the robot's motion, moving freely in any direction. However, they are technically difficult to implement because dirt easily jams the mechanism.

2.4 Robot Configurations and Maneuverability

Robot mobility is defined by the arrangement of these wheels. A robot's design is always a trade-off between **maneuverability** (ability to move freely) and **controllability** (ease of controlling that movement).

- **Differential Drive:** The most common indoor robot configuration (e.g., Roomba). It uses two independently driven wheels and one or more passive caster wheels for stability.
 - **Motion:** It steers by varying the speed of the two drive wheels. If both move at the same speed, it goes straight; if they move in opposite directions, it turns on the spot (zero turning radius).
- **Synchro Drive:** Uses three or more wheels that are mechanically linked. One motor controls the spin of all wheels, and a second motor controls the steering of all wheels simultaneously.

- **Limitation:** The robot body cannot change its orientation (heading). It can move in any direction (x, y) , but the robot always faces the same way.
- **Ackermann Steering (Car-like):** Uses two rear drive wheels and two front steerable wheels. It has high stability but low maneuverability (non-holonomic). It cannot move sideways or turn on the spot, requiring complex maneuvers (like parallel parking) to reach specific poses.
- **Omnidirectional Robots:** Use Swedish or spherical wheels to move instantaneously in any direction (x, y) and rotate (θ) simultaneously. While highly maneuverable, they are complex to control and suffer from uneven vibration and traction issues.

2.5 Stability and Environment Interaction

- **Static Stability:** To be statically stable without active control, a robot needs at least three contact points, and the center of mass must fall within the polygon formed by these points.
- **Suspension:** With three wheels, contact is guaranteed. With four or more wheels, suspension is required to ensure all wheels maintain ground contact, especially on uneven terrain.
- **Passive Adaptation (The Shrimp Robot):** Complex navigation does not always require sensors. The “Shrimp” robot uses a mechanical design with a bogie system and spring-loaded wheels to passively climb obstacles up to two times its wheel radius without needing to “perceive” the obstacle or calculate a path.

Chapter 3

Kinematics of mobile robots

3.1 Kinematics and Coordinate Frames

To control a robot, we must map its internal movements to the real world.

3.1.1 Reference Frames

- **World Frame (W):** Fixed to the environment (e.g., a room).
- **Robot Frame (R):** Attached to the moving robot.

The goal of kinematics is to calculate the transformation between W and R —mapping the robot’s local perception to the global map.

3.1.2 Motion Constraints (Holonomic vs. Non-Holonomic)

A rigid body on a plane has 3 DOF (translation in x, y and rotation θ).

- **Holonomic:** A robot that can control all 3 DOF simultaneously (e.g., an omnidirectional robot can move sideways instantly).
- **Non-Holonomic:** A robot that cannot instantaneously move in certain directions. A car is non-holonomic; it has 3 DOF in the world but only 2 controllable DOF (throttle and steering), preventing it from moving sideways without maneuvering.

3.1.3 Dead Reckoning

This is the process of estimating the robot’s current position based solely on internal sensors (like wheel encoders) without external references (like GPS). It is prone to accumulating errors (drift) over time.

3.2 Differential Drive Math

For a differential drive robot with wheel width W , we calculate the path based on the velocities of the right (v_R) and left (v_L) wheels:

- **Straight Line:** $v_L = v_R$ (Radius $R = \infty$).
- **Rotation on Spot:** $v_L = -v_R$.
- **Curved Motion:** The robot moves around a point called the **ICC** (**I**ntantaneous **C**enter of **C**urvature). The radius of this turn R is determined by the difference in wheel speeds:

$$R = \frac{W(v_R + v_L)}{2(v_R - v_L)}$$

Chapter 4

Robot Perception

4.1 Robot Perception and Sensors

4.1.1 The Challenge of Perception

The Paradox of Intelligence

In robotics, “easy problems are hard and hard problems are easy”. Abstract tasks like playing chess (closed world, complete information) are computationally easy, whereas perception tasks that humans perform unconsciously (like seeing and walking) are extremely difficult for robots.

Open vs. Closed Worlds

Perception is difficult because the real world is an “open world” with dynamic events, occlusions, and incomplete information, unlike the static, fully defined “closed world” of a chessboard.

Current Limitations

While technology has advanced in “seeing” (detecting objects via Deep Learning), robots still struggle with “feeling” or understanding the context and semantics of a situation (e.g., recognizing a dangerous traffic situation versus a safe one).

4.2 The Perception Pipeline: From Data to Semantics

The process of robot perception involves compressing raw data into abstract concepts:

- **Raw Data:** The lowest level consists of numbers (vectors, matrices) coming directly from sensors, such as pixel arrays or range readings.
- **Features:** Algorithms extract patterns from raw data, such as lines, corners, or colors. This often requires imposing a model (e.g., fitting a line to a set of points).
- **Objects:** By combining features, the robot identifies entities (e.g., recognizing a door or a human).
- **Semantics and Context:** The highest level involves reasoning about the relationships between objects (e.g., a door connects two rooms) and understanding the situation or place.
- **Sensor Fusion:** This refers to combining data from multiple sensors (often multimodal, like cameras and lasers) to create a more reliable “percept” or world model.

4.3 Sensor Taxonomy

Sensors are classified based on what they measure and how they interact with the environment:

4.3.1 Proprioceptive vs. Exteroceptive

- **Proprioceptive:** Measures the internal state of the robot (e.g., battery level, wheel encoders, joint angles, internal temperature).
- **Exteroceptive:** Measures the external environment (e.g., cameras, laser scanners, distance sensors).
- **Exproprioceptive:** Measures the relationship between the robot’s body and the environment (e.g., a camera on an arm guiding a gripper toward an object).

4.3.2 Active vs. Passive

- **Active:** The sensor emits energy into the environment and measures the reaction (e.g., sonar, laser range finders, active optical beacons).
- **Passive:** The sensor receives energy already present in the environment (e.g., cameras, microphones, tactile sensors).

Note on GPS: While the GPS system (satellites) is active, the receiver on the robot is typically considered a passive sensor because it only receives signals.

4.4 Characterizing Sensor Performance

Real-world measurements are always error-prone and uncertain.

4.4.1 Key Metrics

- **Dynamic Range:** The ratio between the minimum and maximum measurable values, usually expressed in decibels.
- **Resolution:** The minimum difference between two values that the sensor can detect.
- **Bandwidth:** The speed or frequency at which the sensor can provide updated readings.
- **Sensitivity:** The ratio of output change to input change. Sensors can suffer from cross-sensitivity, where they react to unintended environmental changes (e.g., a light sensor affected by temperature).

4.4.2 Accuracy vs. Repeatability

- **Accuracy:** How close a measurement is to the true value (reference standard).
- **Repeatability (Precision):** The ability to reproduce the same result under the same conditions multiple times.

The Target Analogy: High accuracy means hitting the center of the target; high repeatability means all shots land close together, even if they are far from the center.

4.4.3 Industrial Application

Industrial robots typically have high repeatability but low accuracy. They are programmed by “teaching” (saving a specific posture) rather than by geometric coordinates, ensuring they return to the exact same spot even if the coordinate calculation is imperfect.

4.5 Motion and Heading Sensors (Dead Reckoning)

Dead reckoning is the process of estimating position by integrating internal motion measurements over time, though it is prone to accumulating drift.

- **Wheel Encoders:** Proprioceptive sensors attached to motor axes. They use optical gratings to count transitions and calculate wheel rotation speed and distance. Quadrature encoders can also detect direction.
- **Compass:** Uses the earth's magnetic field to determine absolute heading. It is often unreliable indoors due to interference from power lines and metallic structures.
- **Gyroscopes:** Measure angular velocity to estimate heading changes.
 - **Mechanical:** Rely on a spinning mass maintaining its axis of rotation (momentum). They suffer from drift over time.
 - **Optical:** Use the Sagnac effect with laser beams traveling in opposite directions. They are more precise than mechanical ones but still drift.
- **Accelerometers:** Measure proper acceleration using a spring-mass system or MEMS technology (capacitive changes). They are used to detect tilt (gravity) or dynamic motion.
- **Inertial Measurement Unit (IMU):** A closed system combining accelerometers, gyroscopes, and magnetometers to estimate orientation and motion. Data is often fused using a Kalman Filter.

4.6 Tactile and Contact Sensors

- **Bumpers:** Simple, passive switches that detect physical contact. They are cheap but provide poor localization of the object.
- **Whiskers:** Bio-inspired sensors that can provide information about proximity and object texture.
- **Robot Skin (RoboSkin):** Advanced capacitive sensors organized in triangular modules that cover the robot's body. These allow collaborative robots ("cobots") to detect exactly where and how hard they are touched, ensuring safety during human-robot interaction.

4.7 Technology Drivers

New sensor technologies often drive revolutions in robotics capabilities:

- **Kinect:** Introduced cheap RGB-D (color + depth) sensing, solving the difficulty of measuring the third dimension.
- **Laser Time-of-Flight:** Previously expensive ($\approx 50,000$ euros), these sensors are now affordable enough for household vacuums, providing reliable distance measurements compared to noisy sonar.
- **Soft Actuation:** Emerging technologies in soft grippers and artificial muscles are opening new applications.

4.8 Advanced Proprioception and Exteroceptive Sensing

4.8.1 Inertial Measurement and Dead Reckoning

- **The IMU (Inertial Measurement Unit):** This single sensor combines three gyroscopes, three magnetometers, and three accelerometers to estimate the robot's motion in 6 degrees of freedom (3 rotational and 3 positional axes).
- **Dead Reckoning:** This is the process of estimating position solely by integrating internal measurements (speed, heading, acceleration) without external references, similar to navigating a ship at sea with only a compass.
- **Drift and Uncertainty:** Dead reckoning is prone to accumulated errors over time (drift). For example, a car navigation system in a tunnel uses dead reckoning when GPS is lost; it estimates position based on speed and heading, but errors grow until the car exits the tunnel and receives a corrective external signal.
- **Sensor Fusion:** To combat noise and uncertainty, robots use sensor fusion. This involves combining redundant data (to correct noise) or complementary data (different distinct features) to avoid “false positives” (detecting non-existent obstacles) or “false negatives” (missing real obstacles).

4.8.2 Contact and Tactile Sensors

These are passive sensors that detect interaction only upon physical touch, covering the closest range of perception.

- **Bumpers and Whiskers:**

- **Whiskers:** Simple switches triggered by moving a wire.
- **Bumpers:** Used in vacuum robots, these typically have two switches. Triggering both indicates a frontal collision, while triggering one indicates a side collision, allowing the robot to navigate corners.

- **Tactile Sensors:** Based on capacitors embedded in a soft substrate. When pressed, the substrate deforms, changing the capacitance to detect touch.

- **Robot Skin (Artificial Skin):** To cover large areas (like a humanoid or industrial arm), sensors are organized into a tessellation of triangular modules (e.g., Sai skin). Each module processes data locally and sends it via a bus. Advanced skins can integrate temperature, proximity, and acceleration sensors to mimic human sensitivity.

- **Safety in Collaborative Robots:** Tactile and proximity sensors are critical for “cobots.” Instead of relying on measuring motor current (torque) to detect a collision—which requires a forceful impact—skin sensors allow the robot to detect contact immediately or even pre-collision, enabling faster but safe operation.

4.8.3 Proximity Sensors (Infrared)

- **Principle:** These active sensors emit near-infrared light (LED) and measure the reflection to detect objects within a short range (1–20 cm).

[Image of infrared proximity sensor diagram]

- **Limitations:**

- **Absorption:** Dark or black objects absorb infrared light, yielding no reflection.
- **Cross-talk:** They can be blinded by sunlight (which contains IR) or interference from other sensors.
- **Specular Reflection:** On mirror-like surfaces, light reflects away from the sensor rather than bouncing back. This creates a “blind spot” where the robot thinks the path is clear, potentially leading to collisions.

- **Spectrum Distinction:** It is important to distinguish Near Infrared (used for proximity and night vision) from Thermal Infrared (Far Infrared), which is used to detect heat/temperature.

4.8.4 Triangulation and Structured Light

- **Triangulation:** Distance is measured by projecting a collimated beam (laser point or line) and observing where it falls on a camera sensor. The position of the dot on the image plane correlates geometrically to the distance of the object.
- **Structured Light:** Instead of a single point, a complex pattern (lines or grids) is projected. By analyzing the deformation of this pattern on surfaces, the robot can reconstruct the 3D shape of the environment.
- **3D Cameras:** Devices like the Microsoft Kinect (v1) use an infrared projector and camera to create a dense depth map (RGB-D) of the scene using this principle.

4.8.5 Time of Flight (ToF) Sensors

These sensors measure distance by calculating the time (t) it takes for a signal to travel to an object and return.

$$d = \frac{c \cdot t}{2}$$

- **Speed of Propagation:**
 - **Sound (Sonar):** Travels at ≈ 300 m/s (0.3 m/ms). Easy to measure.
 - **Light (Lidar/ToF Cameras):** Travels at $\approx 300,000$ km/s (0.3 m/ns). Requires nanosecond-precision timing.
- **Ultrasonic Sensors (Sonar):** Cheap and effective for short ranges (e.g., car parking sensors).
 - **Problems:** They have a wide emission cone ($\sim 30^\circ$), leading to poor angular resolution. They also suffer from multipath errors (sound bouncing off multiple walls before returning) and specular reflections (sound bouncing off smooth walls away from the sensor).
- **GPS:** Operates on the ToF principle using radio signals from satellites. It requires extremely precise time synchronization (atomic clocks) but fails indoors or in tunnels (multipath problems).

4.9 Sensor Technologies in Mobile Robotics (Paper 1)

4.9.1 Definition and Classification

- **Sensor vs. Transducer:** A transducer is a device that converts one form of energy into another (e.g., mechanical to electrical). A sensor is a receiver transducer that measures a physical parameter (like light or sound) and converts it into a signal. An actuator is an emitter transducer that injects energy into the environment.
- **Excitation Source:**
 - **Passive Sensors:** Do not require an external power supply to generate an excitation signal; they only receive energy already present (e.g., a camera receiving light, a microphone).
 - **Active Sensors:** Require power to generate their own excitation signal and measure the reaction (e.g., Sonar or LiDAR emitting pulses).
- **Measurement Domain:**
 - **Proprioceptive:** Measure internal robot states (e.g., battery level, wheel speed via encoders).
 - **Exteroceptive:** Measure the environment and the robot's interaction with it (e.g., distance to an obstacle).
- **Key Performance Metrics:**
 - **Resolution:** The smallest detectable change in the input signal.
 - **Precision (Repeatability):** The ability to reproduce the same result for a steady signal (variance).
 - **Accuracy:** How close the measurement is to the true value (correctness).
 - **Dynamic Range:** The ratio between the largest and smallest measurable values, usually expressed in decibels.

4.9.2 Basic Components (Transducers)

- **Force and Deformation:**
 - **Piezoelectric:** Generates voltage when deformed (reversible). Used in accelerometers and sonars.

- **Capacitive:** Measures force via changes in capacitance between plates. Used in MEMS accelerometers and microphones.
- **Light:**
 - **CCD vs. CMOS:** Both are imaging sensors. CCDs shift charges to a central amplifier (sequential), while CMOS pixels have individual amplifiers (faster, common in consumer electronics).
- **Magnetic:**
 - **Hall Effect:** A conductor generates a voltage when traversed by a current in a magnetic field. Used in encoders and compasses.

4.9.3 Proprioceptive and Contact Sensors

- **Bumpers:** Simple contact switches used for safety and collision detection.
- **Encoders:** Measure angular position/velocity of wheels for odometry.
 - **Incremental:** Counts “ticks” (relative motion). Quadrature encoders use two signals shifted by 90° to detect rotation direction and increase resolution.
 - **Absolute:** Provides a unique binary code for every specific angle, retaining position even after power loss.

4.9.4 Global Localization Systems

- **GNSS (Outdoor):**
 - **GPS:** Uses trilateration of satellite signals. Standard accuracy is 5–15m.
 - **RTK (Real-Time Kinematic):** Improves accuracy to centimeter-level by using a base station and measuring carrier wave phases.
- **UWB (Indoor):**
 - **Ultra-Wideband:** Uses short energy pulses for indoor positioning. It uses TDOA (Time Difference of Arrival) or TOF (Time of Flight) between a mobile “tag” and fixed “anchors”. It is more accurate than WiFi/Bluetooth but requires infrastructure.

4.9.5 Inertial and Heading Sensors (IMU)

- **Accelerometers:** Measure linear acceleration. MEMS versions often use capacitive plates on a moving frame.
- **Gyroscopes:** Measure rotational rate.
 - **MEMS (Vibrating Structure):** Exploit the Coriolis effect on a vibrating mass.
 - **FOG (Fiber-Optic):** Use the Sagnac effect (light interference) for high accuracy but are expensive.
- **Compasses:**
 - **Fluxgate:** High accuracy devices using magnetic saturation coils.
 - **Hall Effect:** Common, cheaper implementation using perpendicular magnetometers.

4.9.6 Vision Systems (Cameras)

- **Digital Cameras:**
 - **Pinhole/Thin Lens Model:** Describes how 3D points project onto a 2D sensor plane. A lens is required to focus light rays.
 - **Bayer Filter:** A grid of R, G, and B filters over pixels allows a single sensor to capture color images via interpolation.
- **Omnidirectional Cameras:** Capture 180°–360° views.
 - **Catadioptric:** Uses a camera pointing at a curved mirror (parabolic/hyperbolic).
 - **Polydioptric:** Uses an array of multiple overlapping cameras.
- **Event Cameras:** Asynchronous sensors that only output data (events) when pixel brightness changes. They have very high dynamic range and low latency, making them ideal for high-speed motion.

4.9.7 Ranging and Depth Sensors

- **Sonar (Ultrasonic):** Uses sound waves (TOF) to measure distance. Cheap but suffers from wide emission cones (low angular resolution) and specular reflections.

- **LiDAR (Light Detection and Ranging):**
 - **Continuous Wave (Phase Shift):** Measures phase difference of a continuous beam.
 - **Pulse-based (TOF):** Measures round-trip time of laser pulses. Requires nanosecond timing.
 - **Solid-state LiDAR:** Uses Optical Phased Arrays (OPA) to steer beams without moving mechanical parts.
- **Radar:** Uses radio waves (FMCW). Lower resolution than LiDAR but robust to rain, fog, and dust. Can measure velocity directly via the Doppler effect.
- **ToF Cameras:** Illuminate the whole scene with IR light to get a depth map in one shot. Prone to “multipath interference” (light bouncing off multiple surfaces).
- **Stereo Vision:**
 - **Passive:** Uses two cameras (baseline separation) to calculate depth via triangulation (finding corresponding points). Fails on untextured surfaces.
 - **Active Stereo:** Projects a pattern (texture) to allow matching even on smooth walls.
- **Structured Light:**
 - **Spatial Encoding:** Projects a random dot pattern (e.g., Kinect v1). Good for dynamic scenes.
 - **Temporal Encoding:** Projects a sequence of changing lines/fringes. Higher accuracy but requires the scene to be static.
- **RGB-D:** Combines a color camera with a depth sensor (ToF, Stereo, or Structured Light) to provide colored point clouds.

Chapter 5

Localization and mapping

5.1 Mapping, Navigation, and Advanced Sensing

5.1.1 Advanced Range Sensing (Lidar, Radar, GPS)

To build maps, robots need precise distance measurements.

Laser Range Finders (Lidar)

- **Mechanism:** Lidar uses electromagnetic energy (infrared light) to measure distance via Phase Shift or Time of Flight (ToF) principles. Since light travels extremely fast, phase shift is often used for precision in short-range sensors.
- **2D vs. 3D:** A standard 2D Lidar uses a rotating mirror to steer a single beam on a plane. 3D Lidar (often used in autonomous cars) uses multiple stacked lasers to scan the environment vertically and horizontally, creating a dense point cloud.
- **Limitations:** Lidar struggles in rain, fog, or dust because the light is absorbed or scattered.

Radar

Uses radio waves rather than light. While it has lower resolution than Lidar, it is robust in adverse weather (rain, fog) and is standard in automotive adaptive cruise control.

GPS (Global Positioning System)

- **Operation:** A system of satellites that transmit synchronized time signals. The receiver (which is passive) calculates position via trilateration (intersecting spheres of distance from at least 3-4 satellites).

- **Limitations:** GPS suffers from “Canyoning” (signals blocked by tall buildings), Multipath errors (signals bouncing off walls), and fails completely indoors or under dense tree cover (leaves absorb frequencies due to water content).

5.2 The Mapping Problem: Metric vs. Topological

To navigate, a robot asks: Where am I? Where am I going? How do I get there? The answer depends on the map type.

5.2.1 Metric Maps (Quantitative)

- **Analogy:** Like a city surveyor’s blueprint or an atlas.
- **Characteristics:** Represents the world using Cartesian coordinates, precise distances, and angles. It places the robot in a specific (x, y, θ) location.
- **Use Case:** Essential for precise obstacle avoidance and local motion planning.

5.2.2 Topological Maps (Qualitative)

- **Analogy:** Like a subway (metro) map.
- **Characteristics:** Represents the world as a Graph. Nodes are “Distinctive Places” (Landmarks), and edges are the connectivity between them. It “forgets” exact distances to focus on relationships (e.g., “The kitchen is connected to the hallway”).
- **Use Case:** Efficient for high-level path planning, symbolic reasoning, and long-distance travel.

5.2.3 Hybrid Maps

Most modern robots use a hybrid approach: a Topological map for global planning (Room A to Room B) and local Metric maps (grids) for moving safely within the room.

5.3 Topological Navigation and Landmarks

5.3.1 Landmarks

These are the anchors of topological maps. To be useful, a landmark must be Distinctive, Unique, and Perceivable from different viewpoints (e.g., a specific corner, a door, or a

tower).

- **Natural vs. Artificial:** Robotocists prefer Natural Landmarks (features already present, like building corners). Artificial Landmarks (QR codes, beacons) make sensing easier but require engineering the environment, which is costly and not always feasible.

5.3.2 Distinctive Places & Hill Climbing

To reduce position uncertainty without GPS, robots use Hill Climbing control laws. Upon detecting a landmark (like a corner), the robot moves to maximize the “distinctiveness” of that feature (e.g., equidistant from walls), “resetting” its position error to zero.

5.3.3 Symbol Grounding

A critical concept in AI. The Topological map deals with abstract Symbols (Node A, Node B). To move the robot, these symbols must be “grounded” in the physical world via Control Laws (behaviors like “follow corridor” or “move through door”) linked to the graph edges.

5.4 Configuration Space (C-Space)

To simplify path planning in metric maps, we transform the robot’s physical reality.

- **Definition:** C-Space reduces the robot to a single point. To do this, we “grow” or inflate the obstacles by the robot’s radius.
- **Dimensions:** For a robot moving on a flat plane, C-Space is usually 3-Dimensional (x, y position and θ rotation). The robot effectively navigates through a “tunnel” of free configurations.
- **Representations:**
 - **Occupancy Grids:** The world is discretized into a grid of cells (Occupied/Free). Simple but suffers from Digitization Bias (if an object covers 1% of a cell, the whole cell is marked occupied) and high memory usage.
 - **Generalized Voronoi Graphs (GVG):** A skeleton map where paths are generated equidistant from all obstacles. This maximizes safety but isn’t always the shortest path.
 - **Meadow Maps:** Represent free space using convex polygons.

5.5 3D Mapping Representations

Since the real world is 3D, 2D maps often fail (e.g., a robot seeing table legs but crashing into the tabletop).

- **Point Clouds:** Raw, dense collection of points. High precision but requires unbounded memory.
- **3D Voxel Grids:** Extends the 2D grid into 3D cubes (Voxels). Constant access time but extremely memory intensive.
- **Octrees (e.g., OctoMap):** A recursive tree structure. A large cubic volume is divided into 8 smaller cubes only if occupied. This is highly memory-efficient for 3D mapping.
- **Elevation Maps (2.5D):** A 2D grid where each cell stores a height value. Efficient, but cannot represent overhangs (bridges, tunnels). Multi-Level Surface (MLS) maps solve this by allowing multiple height patches per cell.

5.6 Simultaneous Localization and Mapping (SLAM)

5.6.1 The Hierarchy of Localization Problems

Before addressing mapping, one must understand localization—estimating the robot’s state. There are three levels of difficulty in this problem:

- **Position Tracking:** The initial robot position is known (given by an oracle). The robot tracks its position over time by correcting errors as it moves. This is a local problem.
- **Global Localization:** The robot has no prior knowledge of its location (e.g., it wakes up in an unknown room). It must determine its position from scratch. This is computationally expensive, so once localized, the robot usually switches to tracking.
- **The Kidnapped Robot Problem:** The most challenging scenario. A well-localized robot is physically moved to a new location without being told (teleported). The robot must realize its current localization is wrong and re-localize globally.

5.6.2 The SLAM Problem Definition

- **Definition:** SLAM stands for Simultaneous Localization And Mapping. It is the process where an autonomous robot moves through an unknown environment to incrementally build a consistent map while simultaneously estimating its location within that map.
- **The Chicken-and-Egg Paradox:** A map is required to localize the robot, but the robot's accurate location is required to build the map. Therefore, both must be estimated jointly.
- **Applications:** SLAM is fundamental for autonomy in GPS-denied environments, such as indoors (vacuum robots), underwater, underground (catacombs), or on other planets (Mars rovers).

5.7 The Probabilistic Approach

Modern robotics assumes the Open World paradigm, where sensors and actuators are noisy and models are imperfect. Instead of exact positions, we estimate a **Belief**—a probability density function (PDF) over the state space.

5.7.1 Bayes Filter

The recursive algorithm used to update belief. It consists of two steps:

1. **Prediction:** Updates the belief using the Motion Model (actuators/odometry). This operation generally increases uncertainty (flattens the probability curve).
2. **Correction (Update):** Updates the belief using the Sensor Model (observations). This reduces uncertainty by re-weighting the belief based on likelihood.

5.8 The Kalman Filter (KF) and Extended Kalman Filter (EKF)

5.8.1 Kalman Filter

An analytical solution to the Bayes Filter. It assumes the system is **Linear** and all distributions are **Gaussian** (defined by a Mean vector μ and Covariance matrix Σ).

- **Operation:** It predicts the new state using matrix multiplication and corrects it using the Kalman Gain (K). K acts as a weighting factor: if the sensor is precise (low uncertainty), K is high, and the system trusts the measurement more than the motion prediction.

5.8.2 Extended Kalman Filter (EKF)

Real-world robot motion (trigonometry) and sensors are **Non-Linear**. The EKF solves this by Linearizing the functions around the current estimate using a first-order Taylor expansion (computing Jacobian matrices G and H).

- **Limitations:** EKF assumes the belief is a single Gaussian (Unimodal). It cannot represent ambiguity (e.g., “I am either in Room A or Room B”).

5.9 Particle Filters

- **Concept:** A non-parametric implementation of the Bayes Filter. Instead of math equations, it represents belief using a set of samples (particles). Each particle represents a hypothesis of the robot’s state.
- **Density:** The density of particles in a region corresponds to the probability of the robot being there. This allows the filter to represent Multi-modal distributions (multiple peaks/hypotheses).
- **Algorithm:**
 - **Sampling:** Generate particles based on the motion model (add noise to each particle).
 - **Weighting:** Assign a weight to each particle based on how well it matches the sensor reading (Sensor Model).
 - **Resampling:** Use a “Roulette Wheel” approach to duplicate high-weight particles and delete low-weight ones, concentrating the belief.

5.10 Solving the SLAM Problem

There are three main paradigms for solving SLAM:

5.10.1 EKF SLAM

Solves Online SLAM (estimates only current pose + map). The state vector includes the robot pose (x, y, θ) and the coordinates of all landmarks (x_i, y_i) .

- **Issues:** The Covariance Matrix grows quadratically with the number of landmarks (N^2), making it computationally expensive for large maps. It also requires perfect Data Association (knowing exactly which landmark you are looking at) to avoid divergence.

5.10.2 Particle Filter SLAM (FastSLAM)

Solves Full SLAM (entire trajectory). Each particle represents a full trajectory hypothesis. It is very effective for grid-based maps and handles ambiguities well.

5.10.3 Graph-based SLAM

The current state-of-the-art.

- **Structure:** It models the problem as a Factor Graph. Nodes represent robot poses and landmarks. Edges represent “springs” or constraints derived from observations and motion.
- **Optimization:** Solving SLAM becomes a Least Squares optimization problem (Maximum Likelihood Estimation). The goal is to find the configuration of nodes that minimizes the “tension” in all springs (minimizes the error between expected and actual measurements).
- **Loop Closure:** A critical concept in Graph SLAM. When a robot recognizes a previously visited location, it adds a “Loop Closure” constraint, which drastically reduces accumulated drift and corrects the entire map.

Chapter 6

Motion Planning

6.1 Motion Planning Fundamentals

- **Definition:** Unlike Path Planning (pure geometry), Motion Planning involves computing a collision-free trajectory (q_{start} to q_{goal}) considering time, dynamics, and kinematics.
- **Approaches:**
 - **Combinatorial (Exact):** Decomposes free space (C_{free}) into cells (e.g., Exact Cell Decomposition, Voronoi Diagrams). It guarantees an optimal solution but is computationally heavy.
 - **Sampling-based:** The standard for high-dimensional spaces. It samples random configurations to build a graph/tree.
 - * **PRM (Probabilistic Roadmap):** Builds a map during a learning phase, then queries it.
 - * **RRT (Rapidly-exploring Random Tree):** Grows a tree from the start toward random points. RRT-Connect grows two trees (from start and goal) to meet in the middle.

These are probabilistically complete (probability of finding a solution $\rightarrow 1$ as time $\rightarrow \infty$).

- **Artificial Potential Fields (APF):** Treats the robot as a particle moving in a force field. The goal applies an attractive force, and obstacles apply repulsive forces.
- **Issue:** Prone to getting stuck in Local Minima.

- **OMPL:** The Open Motion Planning Library is the standard backend for planning in ROS (via MoveIt).

6.2 Human-Robot Collaboration (HRC) Planning

- **The Challenge:** Industrial scenarios are no longer static. Planning must account for human presence, safety, and ergonomics.
- **Dynamic Role Allocation:** In tasks like collaborative transportation, the robot switches between Leader (guiding for precision) and Follower (compliant to human force).
- **Policies:** Allocation can be based on force detection (admittance control), vision (tracking human skeleton/velocity), or distance from the goal.
- **Online Adaptation:** Offline planners (like RRT) generate a nominal path, while online planners (like APF) modify it in real-time to avoid moving obstacles (humans) or align with human intention.

6.3 Geometric Motion Planning and Algorithms (Paper 2)

6.3.1 The Geometric Path Planning Problem

- **Definition (The Piano Mover’s Problem):** The fundamental goal is to find a continuous collision-free path for a robot (rigid body or articulated links) from an initial configuration (q_I) to a goal configuration (q_G) amidst static obstacles.
- **Configuration Space (C -Space):** To simplify planning, the robot’s complex geometry is mapped to a single point in a high-dimensional space called the Configuration Space (C).
 - **Dimensions:** The dimension of C equals the robot’s degrees of freedom (DOF). For example, a rigid body in 2D has a 3D C -space (x, y, θ) , while a rigid body in 3D has a 6D C -space (translation + rotation).
 - **Obstacles (C_{obs}):** The set of all configurations where the robot intersects with workspace obstacles.
 - **Free Space (C_{free}):** The set of configurations where the robot does not collide ($C \setminus C_{obs}$). The planner must find a path entirely within C_{free} .
 - **Topology:** The C -space is often a manifold. For instance, a 2-joint arm has a torus topology ($S^1 \times S^1$), and a 3D rotating body uses $SO(3)$, often represented by unit quaternions (RP^3) to avoid singularities.

6.3.2 Complexity and Completeness

- **Computational Hardness:** The basic motion planning problem is PSPACE-hard, meaning the computational effort grows exponentially with the number of degrees of freedom.
- **The “Black Box” Approach:** Because calculating C_{obs} explicitly is computationally expensive, modern algorithms use a collision detector as a “black box.” The planner proposes a specific configuration (q), and the detector simply returns “True” or “False” regarding collision.
- **Completeness Types:**

- **Complete:** An algorithm that finds a solution if one exists, or reports failure in finite time if none exists (e.g., Canny’s roadmap algorithm). These are often too slow for high-dimensional problems.
- **Resolution Complete:** Guaranteed to find a solution if the resolution of the grid/discretization is fine enough.
- **Probabilistic Completeness:** The standard for sampling-based planners. It guarantees that if a solution exists, the probability of finding it approaches 1.0 as the number of samples approaches infinity. However, it cannot definitively report that no solution exists.

6.3.3 Sampling-Based Planning

These methods avoid exact geometric modeling by sampling random points in C -space to build a connectivity graph.

- **Multi-Query Planners (PRM - Probabilistic Roadmap):** Designed for static environments where the map is built once and queried multiple times.
 - **Construction:** Samples are drawn from C_{free} and added as vertices. The algorithm attempts to connect nearby vertices with simple paths (local planner). If successful, an edge is added.
 - **Result:** A roadmap graph G that captures the connectivity of C_{free} . Queries are solved by connecting start/goal configurations to the roadmap and performing a graph search.
- **Single-Query Planners (RRT - Rapidly-exploring Random Trees):** Designed to solve a specific (q_I, q_G) query immediately without pre-computation.
 - **Algorithm:** The tree grows from q_I . In each step, a random sample $(\alpha(i))$ is chosen. The nearest node in the existing tree is extended toward this sample.
 - **Voronoi Bias:** RRTs naturally explore unexplored space because the probability of selecting a node for expansion is proportional to the volume of its Voronoi region (the empty space around it).
 - **Bidirectional Search:** Efficiency is improved by growing two trees (one from start, one from goal) and attempting to connect them.

6.3.4 Combinatorial and Potential Field Approaches

These are alternative paradigms effective for lower-dimensional problems but less scalable than sampling methods.

- **Combinatorial Roadmaps:**

- **Maximum Clearance:** Retracts paths to be as far from obstacles as possible (Voronoi diagrams).
- **Shortest Path (Visibility Graph):** Connects reflex vertices of obstacles. Paths touch obstacle corners.
- **Cell Decomposition:** Breaks C_{free} into simple geometric shapes (e.g., trapezoids). A path is found by moving between adjacent cells.

- **Potential Fields:**

- **Concept:** The robot is treated as a particle moving in a differentiable field. The goal applies an attractive force, and obstacles apply repulsive forces.
- **Gradient Descent:** The robot “rolls downhill” toward the goal.
- **Local Minima Problem:** The robot may get stuck in a “basin” where the net force is zero but it has not reached the goal (e.g., a U-shaped obstacle). Navigation functions can solve this but are hard to construct.

6.3.5 Planning with Differential Constraints

Real robots have limits on velocity and acceleration (dynamics) or steering constraints (nonholonomic).

- **Terminology:**

- **Nonholonomic:** Constraints on velocity that cannot be integrated into position constraints (e.g., a car cannot slide sideways).
- **Kinodynamic:** Problems involving both kinematic (velocity) and dynamic (force/acceleration) constraints.
- **Phase Space (X):** The search happens in a space combining configuration and velocity ($q, \dot{q}$).

- **Reachability Trees:** Instead of sampling points in space, the planner samples control inputs (actions) over time intervals. Integrating these actions generates a tree of reachable trajectories.

- **Decoupled Approach:** A common simplification is to plan a geometric path first (ignoring dynamics), then compute a velocity profile (time scaling) to satisfy constraints along that path.

6.3.6 Extensions and Variations

- **Time-Varying Environments:** If obstacles move, the state space becomes $C \times T$ (Configuration + Time). Time must strictly increase along the path.
- **Multiple Robots:**
 - **Centralized:** Treat all m robots as one giant system with dimension $N = \sum \dim(C_i)$. This is complete but computationally expensive.
 - **Decoupled:** Plan for each robot individually, then coordinate their timing to avoid collisions. This is faster but sacrifices completeness.
- **Closed Kinematic Chains:** Robots with loops (e.g., two arms holding one object) are constrained to a lower-dimensional manifold. Standard sampling fails because random samples rarely satisfy the loop closure constraint.
- **Algebraic Geometry (CAD):** Cylindrical Algebraic Decomposition provides a theoretical upper bound for solving motion planning exactly using polynomial models. It is doubly exponential in complexity and rarely used in practice, but ensures exact completeness.

Chapter 7

Software Architectures for robotics and robotics paradigms

7.1 Robot Architectures

7.1.1 The Hierarchical Paradigm (Classic AI)

- **The Cycle:** This paradigm, dominant from the late 60s to the 80s, operates on a strict serial sequence: SENSE → PLAN → ACT.
- **Characteristics:** It focuses heavily on “planning.” The robot perceives the world to update an internal global model, reasons upon that model to generate a plan, and then executes it.
- **Examples:**
 - **Shakey (1967):** The first AI robot, developed by SRI. It used the STRIPS algorithm to plan actions based on a logical description of the world (predicates like INROOM or PUSHABLE).
 - **The Monkey and Banana Problem:** A classic test case where a planner must deduce how to move a box to reach a banana. It highlights the reliance on a static, fully described world.
- **Limitations (Why it failed):**
 - **The Closed World Assumption:** It assumes the internal model is perfect and contains everything relevant. It cannot handle new, undefined objects (e.g., a human entering a “blocks world”).

- **The Frame Problem:** It is computationally impossible to mathematically describe how every single action affects every aspect of the world state.
- **Processing Bottleneck:** The cycle is too slow for real-time reaction (e.g., balancing a pendulum), as the robot must complete the sensing and planning phases before moving.

7.1.2 The Reactive Paradigm (Behavior-Based Robotics)

- **The Revolution:** Arising in the mid-80s (notably by Rodney Brooks), this paradigm removes the “Plan” block and the global world model. It connects perception directly to action: SENSE $\rightarrow$ ACT.
- **Key Philosophy:**
 - “The world is its own best model”: Instead of storing a map, the robot senses the environment in real-time.
 - “Intelligence is in the eye of the observer”: Complex behavior (like ants foraging) emerges from simple rules interacting with the environment, not from complex internal reasoning.
- **Bio-Inspiration:**
 - **Grey Walter’s Tortoises (*Machina Speculatrix*):** Early analog robots (1950s) that exhibited complex behaviors (seeking light, recharging) using simple vacuum tube circuits.
 - **Braitenberg Vehicles:** Theoretical agents that produce complex behaviors (aggression, cowardice, love) simply by crossing or uncrossing wires between sensors and motors.
- **Subsumption Architecture:** Proposed by Brooks, this organizes behaviors into layers. Lower layers (e.g., “Avoid Objects”) operate as fast reflexes. Higher layers (e.g., “Wander” or “Explore”) can subsume (inhibit or suppress) the lower ones to achieve complex tasks. It uses Augmented Finite State Machines (AFSM) for implementation.
- **Arbitration:** When multiple behaviors (e.g., “Go to Goal” vs. “Avoid Obstacle”) trigger simultaneously, an arbiter decides the action using strategies like Winner-Take-All (priority-based), Vector Summation (cooperative), or Voting.

- **Emergent Behavior:** Functionalities that are not explicitly coded but appear through interaction. For example, robots in Robocup passing the ball not because of a “pass” code, but because of dynamic role allocation logic.

7.1.3 The Hybrid Paradigm

- **Synthesis:** Neither Hierarchical (too slow) nor Reactive (no global awareness) is perfect. Modern robotics uses a Hybrid Paradigm.
- **Structure:** It keeps reactive behaviors at the bottom for real-time safety and control (close to hardware) but adds a deliberative layer on top for long-term planning and world modeling.
- **Operation:** The planner operates asynchronously, configuring or activating specific behaviors which then run reactively.

7.1.4 The Hybrid Paradigm

- **The Problem:** Purely Deliberative systems (Sense-Plan-Act) are too slow and struggle with the “Frame Problem” (modeling a changing world). Purely Reactive systems (Sense-Act) lack memory, planning, and global goal handling.
- **The Solution:** The Hybrid Architecture combines both approaches to maximize their strengths. It organizes the robot into interacting layers with different time scales and levels of abstraction.
- **Layered Structure:**
 - **Deliberative Layer (Upper):** Handles high-level goals, mission planning, and maintaining a global world model. It operates on a slow time scale (Past, Present, Future).
 - **Reactive Layer (Lower):** Handles low-level sensing and acting (behaviors) for real-time safety and reflex actions. It operates on a very fast time scale (Present only).
 - **Sequencing/Interface Layer (Middle):** This is the critical bridge. It acts as a manager that selects which behaviors to use based on the high-level plan (e.g., “activate Follow Hallway behavior”) and monitors progress to report back to the planner.

Chapter 8

Human Robot-interaction

8.1 Teleoperation, and Shared Intelligence

8.1.1 Teleoperation and Telesystems

- **Definition:** A telesystem involves a human and a robot that are physically separated but interacting to accomplish a task. This is often necessary because AI is good at symbol manipulation but poor at converting raw sensor data into semantic symbols (perception).
- **Components:** A standard telesystem has seven components: Display and Control (at the local/human end), Sensors, Effectors, Power, and Control (at the remote/robot end), connected by Communication.
- **The “Human Out-Of-The-Loop” (OOTL) Problem:** A major risk in automation. When a system is highly automated, the human operator loses “situational awareness.” If the automation fails, the human cannot seamlessly take over because they do not understand the current state (e.g., the Air France accident where pilots could not handle a stall after autopilot disengaged).

8.1.2 Taxonomy of Control and Interaction

How the human and robot share the workload allows for different control paradigms:

- **Direct/Manual Control:** The human closes all loops. They interpret the sensor data and send direct motor commands (e.g., wheel speed). The robot has no intelligence.

- **Supervisory Control:** The human sets high-level goals (directives), and the robot executes them autonomously using its own local feedback loops.
- **Traded Control:** The human and robot take turns controlling the system. For example, the robot drives autonomously on a highway, then says, “You take it from here,” handing control back to the human for city driving.
- **Shared Control:** Both human and robot supply control signals simultaneously. The robot might handle high-frequency stability (e.g., keeping a drone level), while the human handles low-frequency guidance (e.g., moving forward).

8.1.3 The “Horse Metaphor” (H-Metaphor)

To solve the complexity of controlling high-DOF robots (like humanoids or exoskeletons), researchers use the H-Metaphor:

- **Concept:** Controlling a robot should be like riding a horse.
- **Role Allocation:**
 - **The Rider (Human):** Provides high-level guidance (e.g., “Go to that tree,” “Stop”).
 - **The Horse (Robot):** Handles the low-level mechanical complexity. The rider doesn’t tell the horse where to place every hoof or how to balance; the horse does that instinctively.
- **Shared Intelligence:** This moves beyond simple control mixing. If the user commands “Go Right,” but the robot perceives a cliff, the robot (like a horse) should refuse the command or modify it to ensure safety, rather than blindly executing it.

8.1.4 Brain-Machine Interfaces (BMI) and Neuro-Robotics

- **Definition:** An interface that interprets user intention directly from the nervous system (e.g., EEG for brain waves, EMG for muscle activity) without physical movement.
- **Challenges:** BMI signals are noisy, have low information transfer rates (few commands possible), and are complex to decode.
- **Shared Intelligence in BMI:** Because BMI commands are uncertain (probabilistic), robots use “Shared Intelligence.” Instead of blindly executing a

noisy brain command, the robot combines the User’s Probability Distribution (e.g., 60% chance user wants “Left”) with the Environmental Affordances (e.g., “Left is blocked by a wall”). The robot filters the command to execute the most logical action.

- **Application (INTELLEXO Project):**

- **Context:** Lower-limb exoskeletons for disabled users.
- **Approach:** Instead of learning full walking trajectories, the system learns Motor Primitives (basic building blocks of movement) from healthy human demonstrations.
- **Adaptation:** The exoskeleton uses an RGB-D camera to see the environment (e.g., stairs, obstacles) and adapts these primitives in real-time to generate collision-free, physiologically natural gait trajectories.